

Python Dictionary: Comprehensive Guide with Questions

Introduction

A **dictionary** in Python is a mutable, unordered collection of key-value pairs enclosed in curly braces {}. Each key must be unique and immutable (such as strings, numbers, or tuples), while values can be any Python object. Dictionaries are optimized for fast lookups with $O(1)$ average time complexity using hash tables[1].

Key Characteristics:

- Mutable (can add, remove, or modify elements)
 - Unordered (Python 3.7+ maintains insertion order, but order should not be relied upon for logic)
 - Key-based access instead of index-based
 - Hash-based lookup (extremely fast $O(1)$ average case)
 - Keys must be hashable (immutable)
 - Values can be any type, including nested dictionaries or lists
-

Creating Dictionaries

Literal Syntax

Empty dictionary

```
my_dict = {}
```

Dictionary with initial values

```
person = {  
    "name": "Alice",  
    "age": 30,  
    "city": "New York",  
    "marks": [85, 92, 78]  
}
```

Mixed key types (valid, but keys must be immutable)

```
mixed_dict = {  
    1: "integer key",  
    "string_key": 42,  
    (1, 2): "tuple key"  
}
```

Using dict() Constructor

From key-value pairs

```
person = dict(name="Bob", age=25, city="London")
```

From iterable of tuples

```
pairs = [("a", 1), ("b", 2), ("c", 3)]  
my_dict = dict(pairs)  
print(my_dict) # {'a': 1, 'b': 2, 'c': 3}
```

Using zip() to pair lists

```
keys = ["name", "age", "city"]  
values = ["Charlie", 28, "Paris"]  
person = dict(zip(keys, values))  
print(person) # {'name': 'Charlie', 'age': 28, 'city': 'Paris'}
```

Using fromkeys()

Create dictionary with default values

```
keys = ["a", "b", "c"]  
default_dict = dict.fromkeys(keys, 0)  
print(default_dict) # {'a': 0, 'b': 0, 'c': 0}
```

⚠ WARNING: Mutable defaults point to same object

```
keys = ["x", "y", "z"]  
bad_dict = dict.fromkeys(keys, [])  
bad_dict["x"].append(1)  
print(bad_dict) # {'x': [1], 'y': [1], 'z': [1]} ← All modified!
```

✓ CORRECT: Use dictionary comprehension for mutable defaults

```
good_dict = {key: [] for key in ["x", "y", "z"]}
good_dict["x"].append(1)
print(good_dict) # {'x': [1], 'y': [], 'z': []}
```

Dictionary Comprehension

Simple comprehension

```
squares = {x: x**2 for x in range(5)}
print(squares) # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

With filtering

```
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
print(even_squares) # {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

From lists

```
names = ["Alice", "Bob", "Charlie"]
length_dict = {name: len(name) for name in names}
print(length_dict) # {'Alice': 5, 'Bob': 3, 'Charlie': 7}
```

Accessing Dictionary Elements

Direct Access (Bracket Notation)

```
person = {"name": "Alice", "age": 30}
```

Access existing key

```
print(person["name"]) # Alice
```

Accessing non-existent key raises KeyError

```
try:
    print(person["city"]) # KeyError: 'city'
except KeyError:
    print("Key not found!")
```

Using `get()` Method (Preferred)

```
person = {"name": "Alice", "age": 30}
```

Returns value if key exists, None otherwise

```
print(person.get("name")) # Alice  
print(person.get("city")) # None
```

Provide default value

```
print(person.get("city", "Unknown")) # Unknown  
print(person.get("age", 0)) # 30
```

Checking Key Existence

```
person = {"name": "Alice", "age": 30}
```

Using `'in'` operator ($O(1)$ average time)

```
if "name" in person:  
    print("Name exists!")
```

Avoid: if `person["name"]` → raises `KeyError` if missing

Using `'keys()'` method (less efficient than `'in'`)

```
if "city" not in person:  
    print("City key does not exist")
```

Modifying Dictionaries

Adding and Updating Values

```
person = {"name": "Alice", "age": 30}
```

Add new key-value pair

```
person["city"] = "New York"
```

Update existing key

```
person["age"] = 31
```

Update multiple values

```
person.update({"age": 32, "city": "Boston"})
```

```
print(person) # {'name': 'Alice', 'age': 32, 'city': 'Boston'}
```

Using `setdefault()`

```
person = {"name": "Alice"}
```

If key exists, return its value; otherwise add it with default value

```
age = person.setdefault("age", 30)
print(age) # 30
print(person) # {'name': 'Alice', 'age': 30}
```

If key already exists, `setdefault` returns existing value

```
age = person.setdefault("age", 25)
print(age) # 30 (existing value, not 25)
```

Removing Elements

```
person = {"name": "Alice", "age": 30, "city": "Boston"}
```

Using `pop()` - returns value and removes key

```
age = person.pop("age")
print(age) # 30
print(person) # {'name': 'Alice', 'city': 'Boston'}
```

pop() with default value (no KeyError)

```
country = person.pop("country", "USA")  
print(country) # USA
```

Using del (raises KeyError if key missing)

```
del person["city"]  
print(person) # {'name': 'Alice'}
```

Using popitem() - removes and returns last inserted pair

```
key, value = person.popitem()  
print(key, value) # name Alice
```

Clear entire dictionary

```
person.clear()  
print(person) # {}
```

Iterating Over Dictionaries

Iterating Over Keys

```
student = {"name": "Alice", "marks": 92, "grade": "A"}
```

Default iteration (keys only)

```
for key in student:  
    print(key) # name, marks, grade
```

Explicit keys()

```
for key in student.keys():  
    print(f"{key}: {student[key]}")
```

Iterating Over Values

```
student = {"name": "Alice", "marks": 92, "grade": "A"}
```

```
for value in student.values():  
    print(value) # Alice, 92, A
```

Iterating Over Key-Value Pairs

```
student = {"name": "Alice", "marks": 92, "grade": "A"}
```

Using items() (most Pythonic)

```
for key, value in student.items():  
    print(f"{key}: {value}")
```

Output:

name: Alice

marks: 92

grade: A

Dictionary Comprehension During Iteration

```
original = {"a": 1, "b": 2, "c": 3}
```

Create new dict with filtered items

```
filtered = {k: v for k, v in original.items() if v > 1}  
print(filtered) # {'b': 2, 'c': 3}
```

Transform values

```
doubled = {k: v*2 for k, v in original.items()}  
print(doubled) # {'a': 2, 'b': 4, 'c': 6}
```

Common Dictionary Methods

Method	Description	Example
keys()	Returns view of all keys	d.keys()
values()	Returns view of all values	d.values()
items()	Returns view of (key, value) tuples	d.items()
get(key, default)	Safe key access with default	d.get("age", 0)
pop(key, default)	Remove and return value	d.pop("age", None)
popitem()	Remove and return last pair	d.popitem()
setdefault(key, val)	Set key if missing, return value	d.setdefault("age", 30)
update(other)	Merge another dict or iterable	d.update({"age": 31})
clear()	Remove all items	d.clear()
copy()	Shallow copy of dictionary	d2 = d.copy()
fromkeys(keys, val)	Create dict from keys and default	dict.fromkeys(["a"], 0)

Common Pitfalls and Best Practices

Pitfall 1: Using Mutable Objects as Keys

✗ WRONG:

```
my_dict = {[1, 2, 3]: "value"} # TypeError: unhashable type: 'list'
```

✓ CORRECT:

```
my_dict = {(1, 2, 3): "value"} # Use tuple (immutable)
```

Pitfall 2: Modifying Dictionary During Iteration

✗ WRONG:

```
d = {"a": 1, "b": 2, "c": 3}
for key in d:
    if key == "b":
        del d[key] # RuntimeError: dictionary changed size during iteration
```

✓ CORRECT:

```
d = {"a": 1, "b": 2, "c": 3}
```


Create list copy of keys first

```
for key in list(d.keys()):  
    if key == "b":  
        del d[key] # Safe
```

Pitfall 3: Shallow Copy Issues

✗ WRONG:

```
original = {"name": "Alice", "scores": [85, 92]}  
copy = original # Creates reference, not copy  
copy["scores"].append(88)  
print(original) # {'name': 'Alice', 'scores': [85, 92, 88]} ← Modified!
```

✓ CORRECT:

```
original = {"name": "Alice", "scores": [85, 92]}  
copy = original.copy() # Shallow copy (nested lists still shared)  
copy["name"] = "Bob"  
print(original) # {'name': 'Alice', 'scores': [85, 92]} ← Safe
```

For deep copy of nested structures

```
import copy  
deep_copy = copy.deepcopy(original)  
deep_copy["scores"].append(88)  
print(original) # {'name': 'Alice', 'scores': [85, 92]} ← Safe
```

Pitfall 4: KeyError on Missing Keys

✗ WRONG:

```
person = {"name": "Alice", "age": 30}  
print(person["city"]) # KeyError: 'city'
```

✓ CORRECT:

```
person = {"name": "Alice", "age": 30}
```

Use get() with default

```
print(person.get("city", "Not specified")) # Not specified
```

Check before accessing

```
if "city" in person:  
    print(person["city"])
```

Pitfall 5: Overwriting Keys Silently

Last value wins, earlier ones are discarded

```
my_dict = {"name": "Alice", "name": "Bob"}  
print(my_dict) # {'name': 'Bob'}
```

Pitfall 6: Using del Without Checking

✗ WRONG:

```
d = {"a": 1, "b": 2}  
del d["c"] # KeyError: 'c'
```

✓ CORRECT:

```
d = {"a": 1, "b": 2}
```

Use pop() with default

```
value = d.pop("c", None) # Returns None, no error
```

Performance Characteristics

Time Complexity

Operation	Average	Worst Case
Access (d[key])	O(1)	O(n)
Insert (d[key] = val)	O(1)	O(n)
Delete (del d[key])	O(1)	O(n)
Search (key in d)	O(1)	O(n)

Note: Worst case (O(n)) occurs with hash collisions, which are rare in practice[2].

Dictionary Lookup Speed

Dictionary lookups are **extremely fast** due to hash tables. For large datasets, using dictionaries for membership testing is far superior to lists[2]:

Slow approach with lists

```
if item in large_list: # O(n)  
    pass
```

Fast approach with dictionaries

if item in large_dict: # O(1) average
pass

Practice Questions

Easy Questions

1. Create a dictionary from two lists

Write a function that takes two lists of equal length and creates a dictionary mapping elements from the first list to elements from the second list.

2. Count character frequency

Write a function that counts the frequency of each character in a string (case-insensitive, alphabetic characters only) and returns a dictionary sorted by frequency descending.

3. Merge two dictionaries

Write a function that merges two dictionaries. If a key exists in both, sum the values.

Intermediate Questions

4. Invert a dictionary

Write a function that swaps keys and values: if the original dictionary is {"a": 1, "b": 2}, the result should be {1: "a", 2: "b"}. Handle the case where values might not be unique.

5. Nested dictionary operations

Given a dictionary of students with nested marks dictionary: {"Alice": {"Math": 85, "Science": 92}, "Bob": {"Math": 78, "Science": 88}}, compute the average score for each student and return a new dictionary.

6. Deep update nested dictionaries

Write a function that recursively updates a nested dictionary. If both base and update have a dictionary value for the same key, merge them recursively.

7. Filter dictionary by criteria

Write a function that removes all key-value pairs from a dictionary where the value is None, an empty string, or an empty list. Return a new dictionary.

Hard Questions

8. Convert list of dictionaries to dictionary of lists

Convert [{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}] to {"name": ["Alice", "Bob"], "age": [30, 25]}.

9. Build reverse index

Given a dictionary where values are lists of items, create a reverse dictionary mapping each item to a list of keys that contained it:

Input: {"a": [1, 2], "b": [2, 3], "c": [1, 3]}

Output: {1: ["a", "c"], 2: ["a", "b"], 3: ["b", "c"]}

10. Dictionary with tuple keys and coordinate operations

Create a dictionary mapping coordinate tuples (x, y) to values. Given a list of coordinates and weights, compute:

- The point with maximum weight
- Group points by quadrant (I, II, III, IV, or "axis")
- Count points and total weight in each quadrant

Solutions to Practice Questions

Easy Solutions

Question 1: Merge lists to dictionary

```
def lists_to_dict(keys, values):  
    return dict(zip(keys, values))
```

Test

```
keys = ["name", "age", "city"]  
values = ["Alice", 30, "Boston"]  
print(lists_to_dict(keys, values)) # {'name': 'Alice', 'age': 30, 'city': 'Boston'}
```

Question 2: Character frequency (sorted)

```
def char_frequency(s):  
    freq = {}  
    for char in s.lower():  
        if char.isalpha():  
            freq[char] = freq.get(char, 0) + 1  
    # Sort by frequency descending  
    return dict(sorted(freq.items(), key=lambda x: x[1], reverse=True))
```

Test

```
print(char_frequency("Hello World")) # {'l': 3, 'o': 2, 'h': 1, 'e': 1, 'w': 1, 'r': 1, 'd': 1}
```

Question 3: Merge dictionaries with sum

```
def merge_dicts(d1, d2):  
    result = d1.copy()  
    for key, value in d2.items():  
        result[key] = result.get(key, 0) + value  
    return result
```

Test

```
d1 = {"a": 1, "b": 2}
d2 = {"b": 3, "c": 4}
print(merge_dicts(d1, d2)) # {'a': 1, 'b': 5, 'c': 4}
```

Intermediate Solutions

Question 4: Invert dictionary

```
def invert_dict(d):
    result = {}
    for key, value in d.items():
        # Handle non-unique values with lists
        if value not in result:
            result[value] = []
        result[value].append(key)
    return result
```

Test

```
original = {"a": 1, "b": 2, "c": 1}
print(invert_dict(original)) # {1: ['a', 'c'], 2: ['b']}
```

Question 5: Average scores per student

```
def average_scores(students):
    result = {}
    for name, marks in students.items():
        avg = sum(marks.values()) / len(marks)
        result[name] = avg
    return result
```

Test

```
students = {"Alice": {"Math": 85, "Science": 92}, "Bob": {"Math": 78, "Science": 88}}
print(average_scores(students)) # {'Alice': 88.5, 'Bob': 83.0}
```

Question 6: Deep update nested dictionaries

```
def deep_update(base, updates):
    result = {**base}
    for key, value in updates.items():
        if isinstance(result.get(key), dict) and isinstance(value, dict):
            result[key] = deep_update(result[key], value)
        else:
            result[key] = value
    return result
```

Test

```
base = {"a": 1, "nested": {"x": 10}}
updates = {"nested": {"y": 20}, "b": 2}
print(deep_update(base, updates)) # {'a': 1, 'nested': {'x': 10, 'y': 20}, 'b': 2}
```

Question 7: Filter dictionary

```
def filter_dict(d):
    return {k: v for k, v in d.items() if v is not None and v != "" and v != []}
```

Test

```
d = {"a": 1, "b": None, "c": "", "d": [], "e": "value"}
print(filter_dict(d)) # {'a': 1, 'e': 'value'}
```

Hard Solutions

Question 8: List of dictionaries to dictionary of lists

```
def reshape_data(list_of_dicts):
    result = {}
    if not list_of_dicts:
        return result
    for key in list_of_dicts[0].keys():
        result[key] = [d[key] for d in list_of_dicts]
    return result
```

Test

```
data = [{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}]
print(reshape_data(data)) # {'name': ['Alice', 'Bob'], 'age': [30, 25]}
```

Question 9: Reverse index

```
def build_reverse_index(d):
    result = {}
    for key, values in d.items():
        for item in values:
            if item not in result:
                result[item] = []
            result[item].append(key)
    return result
```

Test

```
original = {"a": [1, 2], "b": [2, 3], "c": [1, 3]}
print(build_reverse_index(original)) # {1: ['a', 'c'], 2: ['a', 'b'], 3: ['b', 'c']}
```

Question 10: Coordinate operations with quadrants

```
def analyze_coordinates(coord_dict):
    if not coord_dict:
        return None, {}
```

```
    # Find max weight point
    max_point = max(coord_dict, key=coord_dict.get)

    # Group by quadrant
    quadrants = {}
    for (x, y), weight in coord_dict.items():
        if x > 0 and y > 0:
            quad = "I"
        elif x < 0 and y > 0:
            quad = "II"
        elif x < 0 and y < 0:
            quad = "III"
        elif x > 0 and y < 0:
            quad = "IV"
        else:
            quad = "axis"

        if quad not in quadrants:
            quadrants[quad] = {"count": 0, "total_weight": 0}
        quadrants[quad]["count"] += 1
        quadrants[quad]["total_weight"] += weight

    return max_point, quadrants
```

Test

```
coords = {(1, 2): 5, (-1, 3): 4, (2, -1): 8, (0, 0): 1}
max_pt, quads = analyze_coordinates(coords)
print(f"Max point: {max_pt}, Quadrants: {quads}")
```

References

- [1] CPython Documentation. (2024). The Python Data Model: Dictionaries. <https://docs.python.org/3/tutorial/datastructures.html>
- [2] Glinteco. (2023). Python Dictionary Tips, Tricks, and Best Practices. Retrieved from <https://glinteco.com/en/post/python-dictionary-tips-tricks-and-best-practices/>
- [3] Real Python. (2024). Dictionaries in Python. <https://realpython.com/python-dicts/>
- [4] GeeksforGeeks. (2021). Python Dictionary Methods. Retrieved from <https://www.geeksforgeeks.org/python/python-dictionary-methods/>
- [5] SaidaScience. (2025). Python Dictionary Mistakes You Are Making RIGHT NOW! Retrieved from <https://saidatascience.com/python-dictionary-mistakes-you-are-making-right-now/>
- [6] SPS Anderson. (2025). Python Dictionaries: Your Guide to Key-Value Data Storage. Retrieved from <https://www.spsanderson.com/steveondata/posts/2025-06-18/>